MALWARE STATIC ANALYSIS SCRIPT (LAB)

Lead Analyst: Solomon Mwaura

Date: April 27, 2026

Subject: Development of a Python Script for Malware Static Analysis

1. Executive Summary

This report details the development of `final_analysis.py`, a Python script designed to automate initial static analysis of suspicious files. Developed as part of a lab exercise for a junior malware analyst, the script focuses on extracting readable strings, identifying suspicious indicators (IPs, URLs, Emails), and computing cryptographic hashes. The goal is to improve the efficiency of the malware analysis process by providing a structured report of key findings.

2. Lab Instructions and Script Implementation

The script was developed following a step-by-step process as outlined in the lab instructions, ensuring all specified functionalities are implemented.

2.1. Task 1: Identify the File Types for Analysis

The script is designed to analyze binary files, including common executable and library formats such as `.exe`, `.dll`, and `.bin`. It is tested using a sample binary file (e.g., `sample.exe`).

2.2. Task 2: Set Up Dependencies

The script utilizes standard Python libraries:

- `re`: For regular expressions to extract strings and identify patterns.
- `hashlib`: For computing cryptographic hashes (MD5, SHA-1, SHA-256).
- `json`: For saving the analysis output in a structured JSON format.
- `os`: For file system operations, such as checking file existence and extracting file names.

2.3. Task 3: Review the Starter Script

The provided `final_starter.py` was renamed to `final_analysis.py` and served as the foundation for implementing the required functionalities.

2.4. Task 4: Implement String Extraction

The `extract_strings` function was implemented to extract printable ASCII strings of a minimum length of 4 characters from the binary file data using a regular expression.

```
def extract_strings(file_path):
    try:
        with open(file_path, "rb") as f:
            data = f.read()
        pattern = rb"[ -~]{4,}"
        extracted_bytes = re.findall(pattern, data)
        return [s.decode('ascii', errors='ignore').strip() for s in ex
    except Exception as e:
        print(f"Error extracting strings: {e}")
        return []
```

2.5. Task 5: Define Suspicious Patterns

The `find_suspicious_indicators` function was developed to use regular expressions to search for IP addresses, URLs, and email addresses within the extracted strings.

```
def find_suspicious_indicators(strings):
    suspicious_patterns = {
        "IP Address": r"\b(?:\d{1,3}\.){3}\d{1,3}\b",
        "URL": r"https?://(?:[-\w.]|(?:%[\da-fA-F]{2}))+",
        "Email": r"[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+"
    }
    matches = []
    for s in strings:
        for label, pattern in suspicious_patterns.items():
```

```
            found = re.findall(pattern, s)
            for item in found:
                matches.append({
                    "Type": label,
                    "Indicator": item
                })
    return matches
```

2.6. Task 6: Implement Hash Computation

The `compute_hashes` function was implemented to calculate MD5, SHA-1, and SHA-256 hashes. It processes the file in 4KB chunks to efficiently handle large binary files, updating each hash object with every chunk.

```
def compute_hashes(file_path):
    hash_objects = {
        "MD5": hashlib.md5(),
        "SHA1": hashlib.sha1(),
        "SHA256": hashlib.sha256()
    }
    try:
        with open(file_path, "rb") as f:
            while chunk := f.read(4096):
                for obj in hash_objects.values():
                    obj.update(chunk)
        return {name: obj.hexdigest() for name, obj in hash_objects.i
    except Exception as e:
        print(f"Error computing hashes: {e}")
        return {name: "Error" for name in hash_objects.keys()}
```

2.7. Task 7: Modify analyze_file Function

The `analyze_file` function orchestrates the entire analysis process. It calls the `extract_strings`, `find_suspicious_indicators`, and `compute_hashes` functions, aggregates their results, and saves the final output as `analysis_report.json`.

```
def analyze_file(file_path):
    if not os.path.exists(file_path):
        print(f"Error: File \'{file_path}\' not found.")
        return
    print(f"Starting analysis on: {file_path}...")
    strings = extract_strings(file_path)
    suspicious = find_suspicious_indicators(strings)
    hashes = compute_hashes(file_path)
    result = {
        "Analysis Results": {
            "File Name": os.path.basename(file_path),
            "Full Path": os.path.abspath(file_path),
            "File Hashes": hashes,
            "Indicators Found": suspicious,
            "Total Strings Extracted": len(strings)
        }
    }
    try:
        with open("analysis_report.json", "w") as f:
            json.dump(result, f, indent=4)
        print("\n[+] Analysis complete. Results saved to analysis_report.json")
    except Exception as e:
        print(f"Error saving report: {e}")

if __name__ == "__main__":
    file_to_test = "sample.exe"
    analyze_file(file_to_test)
```

3. Conclusion

The `final_analysis.py` script successfully implements all the requirements of the lab, providing a functional and efficient tool for initial static malware analysis. It effectively extracts strings, identifies key suspicious indicators, and computes cryptographic hashes, all while generating a structured JSON report. This script serves as a foundational component for a junior malware analyst's toolkit, enabling quick triage and preliminary assessment of suspicious binary files.
